Automated Software Engineering (2026) 33:58
https://doi.org/10.1007/s10515-026-00600-5

Generative AI as an infrastructure copilot: automating Infrastructure-As-Code across the DevSecOps lifecycle

Matteo Esposito[2] · Mikel Robredo[2] · Alexander Bakhtin[2] · Davide Taibi[1,2] · Valentina Lenarduzzi[1,2]

Received: 12 October 2025 / Accepted: 6 February 2026
© The Author(s) 2026

Abstract

Practitioners and researchers continuously focus on developing automation strategies to cope with the exponentially demanding need for the timely deployment of software projects in tight release schedules. Such automation techniques include Infrastructure-as-Code (IaC) and the DevOps and DevSecOps cycles. Recent studies investigated generative AI (GenAI) for generating infrastructure as code scripts. However, no studies have focused on using GenAI to generate IaC scripts based on DevSecOps stage artifacts. Different IaC tools serve varied purposes, requiring specific infrastructure setups for different project stages. We envision GenAI models leveraging artifacts from each DevSecOps stage to create and refine IaC scripts. We trust our approach to have an impact on practitioners to leverage it as an automatic copilot for infrastructure design and deployment, and for researchers to build on our vision and future empirical validation.

Keywords GenAI · IaC · DevOps · DevSecOps · Security · Agentic AI

✉ Matteo Esposito
matteo.esposito@oulu.fi

Mikel Robredo
mikel.robredomanero@oulu.fi

Alexander Bakhtin
alexander.bakhtin@oulu.fi

Davide Taibi
taibi@imada.sdu.dk

Valentina Lenarduzzi
lenarduzzi@imada.sdu.dk

[1] University of Southern Denmark, Vejle, Denmark

[2] University of Oulu, Oulu, Finland

Published online: 27 February 2026

1 Introduction

IaC allows practitioners to manage their technology and infrastructural stacks through scripts, making them efficient, scalable, and standardized with fewer threats of human error (Morris 2020). However, human effort is required and spent on creating and working with IaC scripts. Recent experiments with Generative AI (GenAI) were mildly successful in creating, interacting with, and automating IaC (Chanus and Aubertin 2023). Recently, Esposito et al. (2025) proposed a MAPE-K-based framework that leverages agentic AI for autonomous anomaly detection and remediation in microservice environments. To the best of our knowledge, no prior work has proposed an end-to-end, lifecycle-aware approach that synthesizes and iteratively refines Infrastructure as Code by leveraging artifacts produced across DevSecOps (DSO) stages (U.S. Department of Defense, Office of the Chief Information Officer 2021). Moreover, different IaC tools can serve various purposes and coexist in the same project (Rahman et al. 2019). We envision GenAI analyzing each DSO stage artifact and preparing the most fitting infrastructure, e.g., for testing, deployment, or monitoring.

On the one hand, our vision allows practitioners to use a copilot to enable infrastructure design and deployment. More specifically, they can adapt and standardize the artifacts of each stage to better represent the needs of the infrastructure. On the other hand, researchers can build on our results and future empirical validation to improve the state of the art by providing new specialized GenAI tools or artifacts to enhance the quality of IaC-deployed infrastructure.

Paper Structure. In Section 2, we provide theoretical background for the study and our framework, and in Section 3, we present its design. Section 4 presents the roadmap for implementing and empirically validating the proposed framework. In Section 5, we discuss limitations and the ethical implications. In Section 6, we conclude our vision.

2 Background and motivation

The concept of IaC was initiated by C. Fowler, who coined the concept of immutable infrastructure and disposable components, which he considered as the replacement for the traditional mutable infrastructure practice with unmodified servers and components after deployment (Fowler 2013). Such "modern" approach can benefit practitioners by providing simplified operations, continuous deployment, reduced errors, and minimal downtime with cloud reboots, which align closely with modern cloud architecture with the principles of DevOps and CI/CD practices. Many studies have proposed adopting IaC approaches, which are used in various sub-domains of software engineering. For example, IaC can be used to construct benchmark tools to evaluate the performance of native cloud applications (Scheuner et al. 2015). Other studies also proposed that IaC can be adopted to implement DevOps in terms of configuration management, code versioning, and revision management. Artac et al. (2017). Similarly, previous research contributed to automated provisioning, continuous deployment, and microservice implementation (Parnin et al. 2017). IaC is also

used to facilitate software testing and code quality (Ikeshita et al. 2017). Many identified best practices regarding adopting IaC must be followed, where Generative AI can help guarantee and control quality results (Kumara et al. 2021).

For DevOps, the adoption of IaC can provide many critical benefits to practitioners, but also presents a variety of challenges (Guerriero et al. 2019; Rahman 2018). For example, consistency and standardization across various environments are some benefits that can help reduce potential errors in the configuration process (Guerriero et al. 2019). When IaC is adopted to ensure this consistency, automation of infrastructure provisioning will greatly improve deployment efficiency, avoiding unnecessary manual intervention (Esposito et al. 2026b). Furthermore, IaC can also ease the effort for infrastructure scalability with reusable scripts and adaptability to additional required resources. Moreover, IaC scripts can be checked into version control systems such as git. From the management perspective, effective collaboration between distributed teams contributes to cost savings.

On the other hand, the code quality of the scripts is critical for each stage of the DSO process supported by IaC. With generative AI, the scripts can be generated automatically, saving developer effort and guaranteeing quality given the proper guidelines. Developers thus can put more effort into designing software business logic, which has a greater influence on the robustness of the architecture and the perceived software quality of end users.

3 Design

In this section, we present our **vision** on the role of GenAI in automating the generation of IaC scripts within the DSO cycle. It is worth noting that infrastructures can serve different purposes based on the developer's needs during development, testing, and deployment (Rahman et al. 2019). This section takes advantage of the nine stages of DSO (U.S. Department of Defense, Office of the Chief Information Officer 2021; Inc. 2022) and identifies the relevant artifacts for each stage, the security concerns, and the role of GenAI in creating IaC scripts. In the upcoming empirical validation, the ultimate goal is to provide a set of GenAI models and guidelines to automate, as much as possible, the creation of IaC starting from the currently available artifacts, whether it is building a testing environment or a production one. Taking as baseline the DSO model (Inc. 2022), Fig. 1 displays the diagram showing the introduction of GenAI's role into the DSO workflow, as well as a visualized list of identified input artifacts and envisioned IaC automation outcomes. Within the current section, we will expand our vision following each of the stages reported in the DSO model.

3.1 Plan

Artifacts The planning stage is the least automated step within the DSO workflow, but crucial for successful integration. The planning aims at conducting a thorough security analysis and developing a detailed schedule for security testing. The baseline to undergo a human planning process entails artifacts such as user stories, software requirements, and architecture diagrams defining the scope, functionality, and struc-

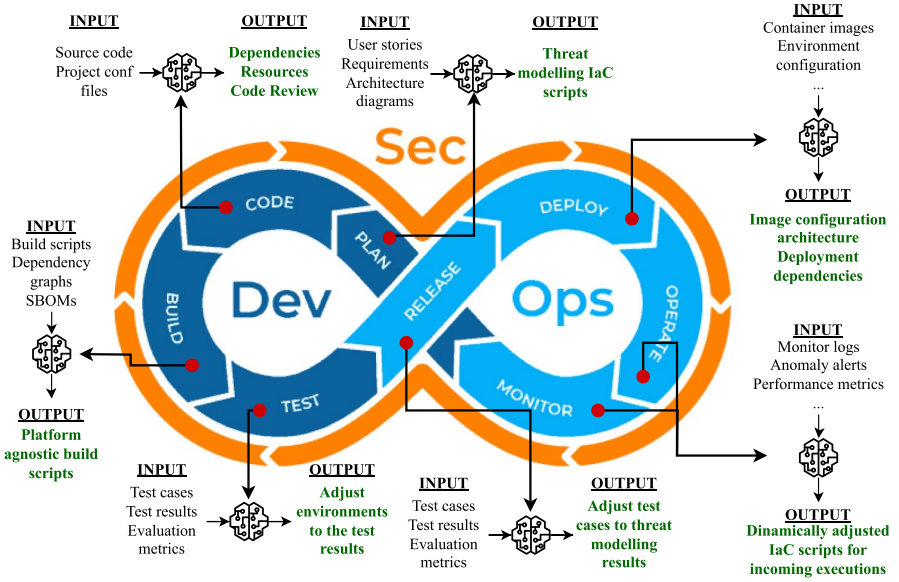

Fig. 1 Expanded DSO model workflow chart describing each of the identified input artifacts and potential IaC script automated outcomes

ture of the software project. Such input artifacts should be synthesized in such format to efficiently enable GenAI models on identifying the key components to automatically define the outcome IaC script. Practitioners could collect the listed components from currently widely employed tools such as IriusRisk [1], Slack [2], as well as well-known tools such as Jira [3].

Security In the planning stage, practitioners should gather security requirements identified within the listed artifacts, and perform threat modelling to identify vulnerabilities early in the design (Ekoramaradhya and Thorpe 2022).

GenAI role Within the present stage, users should feed the GenAI model with the synthesized user stories, software requirements and the rest of input artifacts. Based on these, the model shall sketch and build the infrastructure needed for the development and deployment of the project. For instance, the GenAI model could propose potentially scalable solutions based on non-functional requirements, e.g. determining the necessary cloud services to ensure high availability and disaster recovery, appropriate load-balancing configurations, and automated scaling policies to maintain performance under varying loads. Moreover, detailed user stories could enable the GenAI model to provide multiple IaC script proposals based on the highlighted keywords. For example, given a user story describing the need for a highly available

[1] https://www.iriusrisk.com

[2] https://slack.com

[3] https://www.atlassian.com/software/jira

web application, the GenAI model could propose infrastructure configurations such as multiple availability zones. Finally, leveraging security artifacts such as the threat model, the model could design IaC script handling the averting of potential threats.

3.2 Develop

Artifacts The development (or code) stage within a DSO model involves the security testing of the existing source code. Therefore, potential artifacts in this stage could range from the project's source code to the configuration files, as well as the security reports from already used static security analysis tools (SSAT).

Security In the coding stage, practitioners could use SSAT that are limited to predefined rules to monitor potential bugs, security vulnerabilities, common coding errors such as SQL injection, cross-site scripting, memory leaks, null pointer dereferences (Esposito et al. 2024a), as well as architectural smells (Esposito et al. 2025; Fontana et al. 2017).

GenAI role Within this stage, the GenAI model shall exploit the information from the identified artifacts and extract key contextual information, such as dependencies, required resources, and configurations, to create and modify IaC scripts. This would ensure that any necessary change to the infrastructure, needed to support new features or enable software updates, could be quickly deployed and automatically managed. Moreover, the GenAI model could leverage the output of the employed SSATs to avoid defining insecure or smell-prone IaC, thus acting as an additional code reviewer.

3.3 Build

Artifacts The building stage could provide artifacts such as the developed build scripts based on the existing source code, as well as derived outputs from the build stage, such as dependency graphs or the Software Bill of Materials (SBOM) of a project, for instance.

Security The artifacts considered in this stage are mainly the build output from the developed code in the previous workflow stage. These can include output reports from already existing analysis tools for the build stage of a project. Thus, we could leverage the output from SSATs specialized in compiled code or bytecode (Hutcheson et al. 2024) to catch any bug or vulnerabilities detectable after each build time.

GenAI role Recent industry insights (Guerriero et al. 2019) have shown that heterogeneity, such as *polyglotism* is one of the crucial challenges in IaC automation. For instance, polyglotism involves using multiple programming languages within a single environment and is essential for managing build scripts and dependencies in diverse technical projects. Furthermore, *portability* stands also as a crucial aspect as it ensures that the software product can be easily transferred and executed across different systems and environments without modifications. No previous study has

investigated the proficiency of GenAI in streamlining the automation and creation of IaC from build scripts by detecting various programming languages used in a project, managing cross-language dependencies for enhanced compatibility, and optimizing build strategies to reduce both time and resource usage. Therefore, this direction has the potential to reduce manual effort and minimize errors throughout the build process. Furthermore, a GenAI model could dynamically modify IaC scripts to provision build environments, creating platform-agnostic scripts, and tailoring these environments based on evolving project needs and new dependencies. As projects grow and change, the GenAI model would dynamically continue to refine and optimize these scripts, ensuring optimal performance across various environments.

3.4 Test

Artifacts The test phase is initiated once a build artifact has been successfully built and delivered to staging or testing environments. Within this process, the artifacts providing project context could be the defined test case suite, the already reported test results, as well as the obtained testing evaluation metrics.

Security Within the testing stage, practitioners usually design security test cases to add them to the test case suite. These test cases analyze the system response when it receives carefully crafted unexpected messages or data concerning access, authentication, and accounting (that is, the AAA in security) (Esposito and Falessi 2024; Sönmez and Kiliç 2021). Practitioners should gather the outcome from the employed security test cases to enable dynamic application security based on the generated reports.

GenAI role In the testing stage, practitioners should provide GenAI models with the defined test cases, expected results, report cases, test evaluation metrics, and dynamic application security testing output. The model shall refine the IaC scripts with the required configurations for the test and deployment environments based on the changes observed in the provided artifacts. The application of the updated IaC script shall be automated, such that the outcome from the incoming new builds returns improved security testing reports.

3.5 Release

Artifacts Once the testing stage is undergone, the release stage involves protecting the runtime environment architecture by reviewing the environment configuration. This stage entails a set of diverse components that can serve the GenAI model as context for the required IaC script automation process. Such components could involve software release notes, deployment scripts, version control tags, release checklists, and test reports.

Security During the release stage, practitioners should manage any identified vulnerabilities and implement remediation efforts. In addition, a second threat modeling phase should be performed. Back during the planning stage, the threat model was

broader and focused on all subsequent stages. In contrast, the threat modeling carried out during the release stage is focused on the aftermath of releasing the binaries; therefore, it should consider cloud and serverless security, container security, secrets management, and possible bug bounty hunting (Akgul et al. 2023). These security concerns are evaluated in the following stages.

GenAI role In the release stage, the GenAI should use the release notes to align infrastructure changes with software updates to optimize deployment scripts. Moreover, the model could align the generated IaC scripts with a certain version of the codebase using version control tags. Additionally, practitioners could benefit from using the GenAI model to leverage the new threat model outcomes, release checklists, and test reports to reduce human error during an update in the case of remediation of infrastructure issues.

3.6 Deploy

Artifacts In the deployment phase, and only if the earlier process succeeded, the deployment of the software product happens. Within this context, practitioners can make use of created deployment manifests, container images, as well as Helm charts, environment configuration, and service discovery information generated during the deployment phase.

Security In the deployment stage, tools like Terraform [4], Ansible [5], Puppet [6], or Chef [7] could test the project for known security flows, and consequently execute IaC scripts. Such checks are based on policies or against benchmarks such as the CIS benchmarks (CIS 2018). Additionally, widely used container image scanning tools such as Clair [8], Trivy [9], and Aqua [10] can provide insightful content for the GenAI model.

GenAI role In this stage, the GenAI model should inspect the container images to understand how to configure and prepare the architecture to run them. Moreover, the model could use Helm charts to improve the already existing infrastructure and accommodate more efficient Kubernetes deployment and environment dependencies. Finally, the GenAI model shall leverage the output from the employed security checks to avoid deploying insecure infrastructure, thus refining and fixing IaC scripts for subsequent deployment stages.

[4] https://developer.hashicorp.com/terraform

[5] https://docs.ansible.com

[6] https://www.puppet.com

[7] https://www.chef.io

[8] https://clairproject.org

[9] https://trivy.dev/latest/

[10] https://www.aquasec.com

3.7 Operate, monitor and respond

Artifacts We grouped the Operation, Monitor, and Respond phases given the already significant importance, constant control over these stages has within the DSO workflow. These stages involve the constant monitoring, anomaly detection, as well as performance statistics obtained from the software deployment. From the processes, practitioners could leverage monitoring data, process logs, incident reports, as well as configuration management data, deployment performance metrics, anomaly alerts, environment usage statistics, and performance reports.

Security These stages altogether involve continuous log aggregation and analysis of the running system, security monitoring, real-time threat detection, endpoint security management, user activities, and secret management, as well as software reaction to these observations.

GenAI role The GenAI model shall continuously refine the infrastructure in real-time as the monitoring data becomes available, improving the system performance and resilience to cyber threats. Essentially, the model should act in a proactive style since users could automatically provide anomaly alerts, environment usage statistics, and performance reports in advance. The GenAI model could analyze the provided input with additional security data, such as detected security anomaly logs, to identify potential problems, learn, and sketch updated IaC script solutions. Thus, users could exploit continuous monitoring to dynamically adjust the infrastructure with the resource needs gathered from the real-time data and remediate possible threats by issuing temporary IaC commands and scripts through the GenAI model.

4 Roadmap

This section presents the future work for implementing and empirically validating our proposed vision for GenAI-enabled DevSecOps (see Section 3). Figure 2 presents the envisioned DSO workflow diagram, which visually shows the introduction of the GenAI role component into the architecture. The implementation must leverage real-world projects adopting IaC tools, while ensuring compliance with established **quality, security, and best-practice standards**. Thus, the implementation should be translated into a **proof of concept that integrates GenAI models into the DevSecOps workflow** through stage-specific hooks and triggers, enabling the automated generation and continuous refinement of IaC scripts based on available artifacts. Similarly, the empirical validation should be conducted by **systematically comparing GenAI-generated IaC scripts with those produced by human experts and state-of-the-art automation tools across DevSecOps stages**, assessing aspects such as correctness, security compliance, adaptability to change, and overall effectiveness. Thus, the evaluation will aim to determine whether GenAI can reliably act as an infrastructure copilot, as well as to identify current limitations, open challenges, and directions for future research.

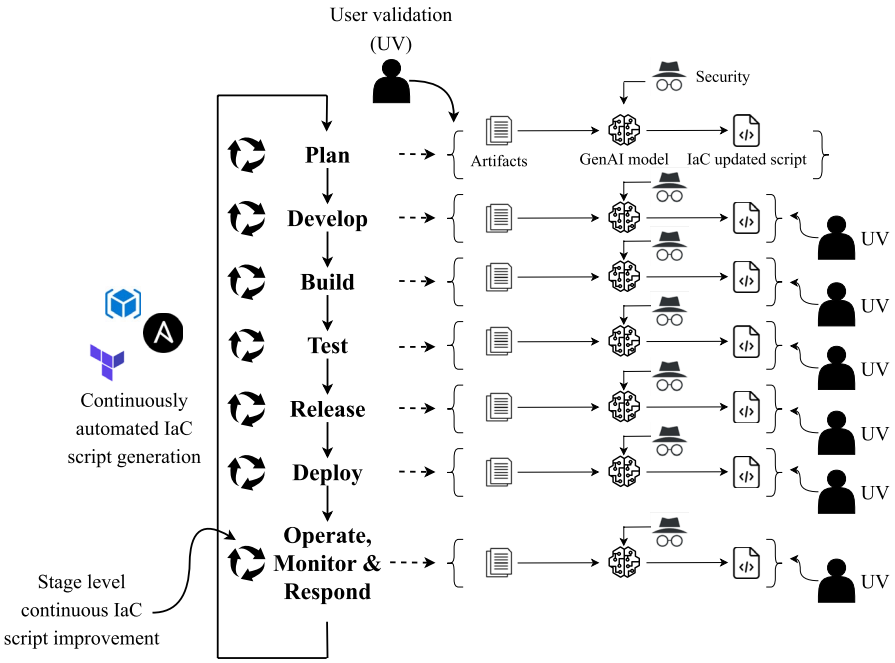

Fig. 2 Envisioned GenAI enabled DSO architecture diagram

5 Limitations and ethical implications

The proposed vision and its design present several limitations and ethical considerations. GenAI models may undergo difficulties with **complex or security-critical infrastructure requirements** (Zhang et al. 2025), as well as limitations in training **data quality** (Siddiq et al. 2024), **scalability**, and **AI explainability** (Esposito et al. 2024b; Amershi et al. 2019), which might lead to inaccurate, insecure, or biased IaC scripts. These risks can potentially occur in large-scale and distributed DevSecOps environments, where scalability, continuous adaptation, and security validation remain challenging (Esposito and Palagiano 2024). To mitigate these issues, in our vision, we emphasize the importance of **data governance**, **modular and incremental integration**, **security validation profiles** (Asare et al. 2023), **explainability mechanisms** (Esteves et al. 2020), and **a human-in-the-loop** (Esposito et al. 2025a) for critical infrastructure decisions. From an ethical perspective, it is crucial to discuss ethical issues arising in all aspects of Software Engineering (Johnson and Menzies 2024). GenAI and agentic AI may introduce concerns related to hallucinations, privacy leakage, backdoor vulnerabilities, and the impact of automation on the software engineering workforce (De Souza et al. 2024; Zhang et al. 2025). Consistent with prior work (Esposito et al. 2024b), we stress the point that **GenAI should be** used as a **valuable assistant**, lifting the heavy work from experts, and **not as expert replacement**.

6 Conclusions

In this work, we presented our vision and how it can potentially provide a paradigm shift in infrastructure setup by integrating GenAI into a DSO cycle to automatically generate and maintain IaC scripts. We proposed a roadmap for implementation and the empirical validation of the proposed pipelines. Our empirical validation aims to compare GenAI-generated IaC scripts with state-of-the-art tools and human experts to understand whether they can be exploited as fundamental tools and or copilots or if they fail at the current stage, therefore highlighting possible weaknesses, literature gaps, and future research direction. Researchers can leverage our paper as a blueprint for paving the way to an extensive empirical validation in this area to support and investigate how specific artifacts can be effectively employed by the models to generate specific infrastructures according to the DSO cycle, thus introducing a consolidated version of artifacts eventually to improve their usability by models.

Finally, we trust our approach to have an impact on practitioners who can leverage it as an automatic copilot for infrastructure design and deployment and for researchers who can build on our results and future empirical validation to improve further and expand the possibilities enabled by it.

6.1 Supplementary information

An extended version of Sections 4 and 5 is available in the Online Appendix [11].

Author contributions Matteo Esposito: Conceptualization, Methodology, Writing — Original Draft Preparation. Matteo envisioned the methodology and contributed significantly to writing the initial manuscript draft. Mikel Robredo: Writing — Original Draft Preparation, Methodology. Mikel contributed to preparing the visualizations and drafting the final manuscript. Alexander Bakhtin: Writing — Original Draft Preparation, Methodology. Alexander contributed to drafting the original manuscript. Davide Taibi: Supervision, Validation, Writing — Review & Editing. Davide led the conceptualization and methodological framing of the research, supervised the entire project, and contributed to reviewing and refining the manuscript. Valentina Lenarduzzi: Supervision, Validation, Writing – Review & Editing. Valentina led the conceptualization and methodological framing of the research, supervised the entire project, and contributed to reviewing and refining the manuscript.

Funding Open Access funding provided by University of Oulu (including Oulu University Hospital). This work has been funded by the Research Council of Finland (grants n. 359861 and 349488 - MuFAno), by Business Finland (grant 6GSoft Akbar et al. 2024), and FAST, the Finnish Software Engineering Doctoral Research Network, funded by the Ministry of Education and Culture, Finland.

Data availability Our study did not generate or analyse any datasets. An extended online appendix is available.

[11] https://doi.org/10.5281/zenodo.18421931

Declarations

Ethical approval Our work did not need ethical approval.

Conflicts of interest We declare that we have no conflict of interests.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Akbar, M.A., Esposito, M., Hyrynsalmi, S., et al.: 6GSoft: Software for Edge-to-Cloud Continuum. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). IEEE Computer Society, Los Alamitos, CA, USA, pp. 499–506 (2024). https://doi.org/10.1109/SEAA64295.2024.00082, https://doi.ieeecomputersociety.org/10.1109/SEAA64295.2024.00082

Akgul, O., Eghtesad, T., Elazari, A., et al.: Bug {Hunters'} perspectives on the challenges and benefits of the bug bounty ecosystem. In: 32nd USENIX Security Symposium (USENIX Security 23), pp. 2275–2291 (2023) https://www.usenix.org/conference/usenixsecurity23/presentation/akgul

Amershi, S., Begel, A., Bird, C., et al.: Software engineering for machine learning: A case study. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, pp. 291–300 (2019) https://doi.org/10.1109/ICSE-SEIP.2019.00042

Artac, M., Borovssak, T., Di Nitto, E., et al.: Devops: introducing infrastructure-as-code. In: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), IEEE, pp. 497–498 (2017) https://doi.org/10.1109/ICSE-C.2017.162

Asare, O., Nagappan, M., Asokan, N.: Is github's copilot as bad as humans at introducing vulnerabilities in code? Empir. Softw. Eng. **28**(6), 129 (2023) https://doi.org/10.1007/s10664-023-10380-1

Chanus, T., Aubertin, M.: Llm and infrastructure as a code use case (2023). arXiv preprint arXiv:2309.01456

CIS.: Cis benchmarks (2018). https://www.cisecurity.org/cis-benchmarks/, accessed: 2025–05-30

Ekoramaradhya, M., Thorpe, C.: A novel devsecops model for robust security in an mqtt internet of things. In: International Conference on Cyber Warfare and Security, pp. 63–71 (2022) https://www.proquest.com/conference-papers-proceedings/novel-devsecops-model-robust-security-mqtt/docview/2681923740/se-2

Esposito, M., Falessi, D.: Validate: A deep dive into vulnerability prediction datasets. Inf. Softw. Technol., p. 107448 (2024) https://doi.org/10.1016/j.infsof.2024.107448

Esposito, M., Palagiano, F.: Leveraging large language models for preliminary security risk analysis: A mission-critical case study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, pp. 442–445 (2024) https://doi.org/10.1145/3661167.3661226

Esposito, M., Falaschi, V., Falessi, D.: An extensive comparison of static application security testing tools. In: International Conference on Evaluation and Assessment in Software Engineering, pp. 69–78 (2024a) https://doi.org/10.1145/3661167.3661199

Esposito, M., Palagiano, F., Lenarduzzi, V., et al.: Beyond words: On large language models actionability in mission-critical risk analysis. In: International Symposium on Empirical Software Engineering and Measurement, pp. 517–527 (2024b) https://doi.org/10.1145/3674805.3695401

Esposito, M., Palagiano, F., Lenarduzzi, V., et al.: On large language models in mission-critical it governance: Are we ready yet? In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 504–515 (2025a). https://doi.org/10.1109/ICSE-SEIP66354.2025.00050

Esposito, M., Robredo, M., Fontana, F.A., et al.: On the correlation between architectural smells and static analysis warnings. Software Qual. J. (2025). https://doi.org/10.1007/s11219-025-09730-7

Esposito, M., Bakhtin, A., Ahmad, N., et al.: Autonomic microservice management via agentic ai and mape-k integration. In: Software Architecture. ECSA 2025 Tracks and Workshops. Springer Nature Switzerland, Cham, pp. 105–118 (2025) https://doi.org/10.1007/978-3-032-04403-7_11

Esposito, M., Li, X., Moreschini, S., et al.: Generative ai for software architecture. applications, challenges, and future directions. J. Syst. Soft., **231**, 112607 (2026b). https://doi.org/10.1016/j.jss.2025.112607,, https://www.sciencedirect.com/science/article/pii/S0164121225002766

Esteves, G., Figueiredo, E., Veloso, A., et al.: Understanding machine learning software defect predictions. Autom. Softw. Eng. **27**(3), 369–392 (2020) https://doi.org/10.1007/s10515-020-00277-4

Fontana, F.A., Pigazzini, I., Roveda, R., et al.: Arcan: A tool for architectural smells detection. In: 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), IEEE, pp. 282–285 (2017) https://doi.org/10.1109/ICSAW.2017.16

Fowler, C.: Trash your servers and burn your code: Immutable infrastructure and disposable components (2013). http://chadfowler.com/blog/2013/06/23/immutable-deployments/

Guerriero, M., Garriga, M., Tamburri, D.A., et al.: Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In: International conference on software maintenance and evolution (ICSME), pp. 580–589 (2019) https://doi.org/10.1109/ICSME.2019.00092

Hutcheson, R., Blanchard, A., Lambaria, N., et al.: Software architecture reconstruction for microservice systems using static analysis via graalvm native image. In: 2024 IEEE International Conference on Software Analysis, pp. 12–22. Evolution and Reengineering (SANER), IEEE (2024) https://doi.org/10.1109/SANER60148.2024.00008

Ikeshita, K., Ishikawa, F., Honiden, S.: Test suite reduction in idempotence testing of infrastructure as code. In: Tests and Proofs: 11th International Conference, TAP 2017, Held as Part of STAF 2017, Marburg, Germany, July 19–20, 2017, Proceedings 11, Springer, pp. 98–115 (2017) https://doi.org/10.1007/978-3-319-61467-0

Inc. VG.: What are the phases of devsecops (2022). https://www.veritis.com/blog/what-are-the-phases-of-devsecops/

Johnson, B., Menzies, T.: Ethics: Why software engineers can't afford to look away. IEEE Softw. **41**(1), 142–144 (2024) https://doi.org/10.1109/MS.2023.3319768

Kumara, I., Garriga, M., Romeu, A.U., et al.: The do's and don'ts of infrastructure code: A systematic gray literature review. Inf. Softw. Technol. **137**, 106593 (2021) https://doi.org/10.1016/j.infsof.2021.106593

Morris, K.: Infrastructure as code. 2nd Edition, O'Reilly Media Inc. (2020) https://www.oreilly.com/library/view/infrastructure-as-code/9781098114664/

Parnin, C., et al.: The top 10 adages in continuous deployment. IEEE Softw. **34**(3), 86–95 (2017) https://doi.org/10.1109/MS.2017.86

Rahman, A.: Characteristics of defective infrastructure as code scripts in devops. In: Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings, pp. 476–479 (2018) https://doi.org/10.1145/3183440.3183452

Rahman, A., Mahdavi-Hezaveh, R., Williams, L.: A systematic mapping study of infrastructure as code research. Inf. Softw. Technol. **108**, 65–77 (2019) https://doi.org/10.1016/j.infsof.2018.12.004

Scheuner, J., Cito, J., Leitner, P., et al.: Cloud workbench: Benchmarking iaas providers based on infrastructure-as-code. In: International Conference on World Wide Web, pp. 239–242 (2015) https://doi.org/10.1145/2740908.2742833

Siddiq, M.L., Roney, L., Zhang, J., et al.: Quality assessment of chatgpt generated code and their use by developers. In: Proceedings of the 21st International Conference on Mining Software Repositories, pp. 152–156 (2024) https://doi.org/10.1145/3643991.3645071

Sönmez, F.Ö., Kiliç, B.G.: Holistic web application security visualization for multi-project and multi-phase dynamic application security test results. IEEE Access **9**, 25858–25884 (2021) https://doi.org/10.1109/ACCESS.2021.3057044

de Souza, C.R., Rodríguez-Pérez, G., Basha, M., et al.: The fine balance between helping with your job and taking it: Ai code assistants come to the fore. IEEE Software (2024) https://doi.org/10.1109/MS.2024.3357787

U.S. Department of Defense, Office of the Chief Information Officer (2021) Devsecops fundamentals guidebook: Devsecops tools and activities. Tech. rep., Department of Defense, accessed: 2025–05-30 https://dodcio.defense.gov/Portals/0/Documents/Library/DevSecOpsTools-ActivitiesGuidebook.pdf

Zhang, R., Li, H.W., Qian, X.Y., et al.: On large language models safety, security, and privacy: A survey. J. Electr. Sci. Technol., p. 100301 (2025) https://doi.org/10.1016/j.jnlest.2025.100301

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.